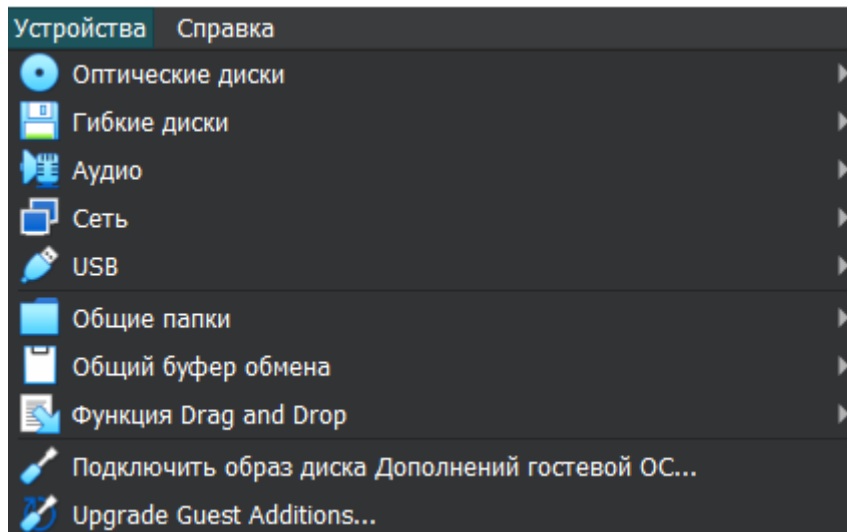


Ввод шестнадцатеричного числа через Debug.

Учебно-тренировочная работа.

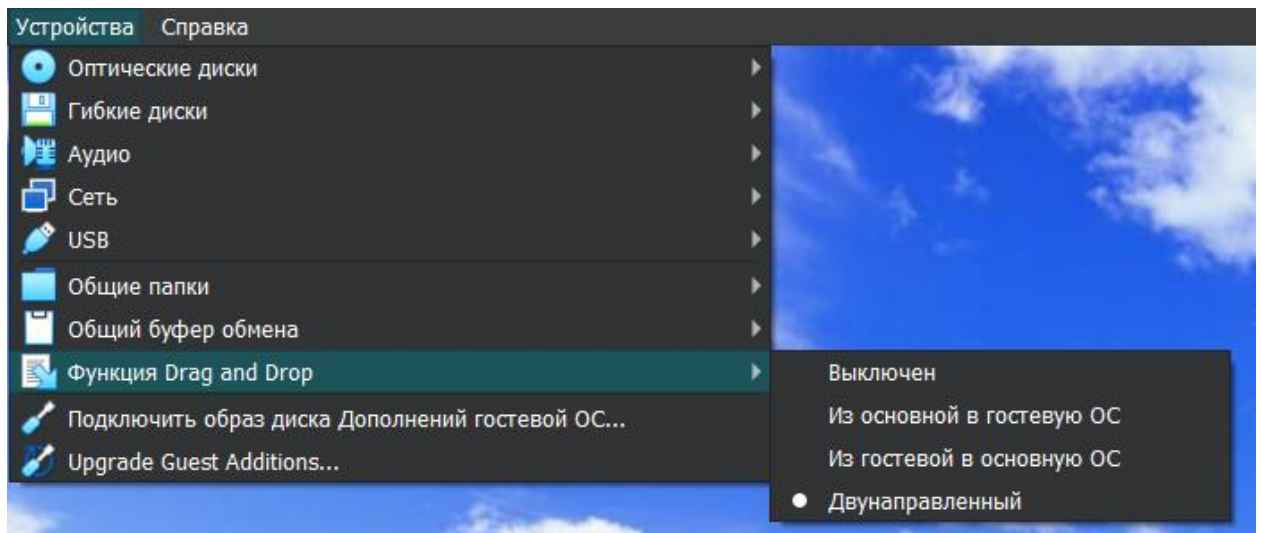
Для того чтобы установить транслятор NASM был подключен образ диска дополнений гостевой ОС.



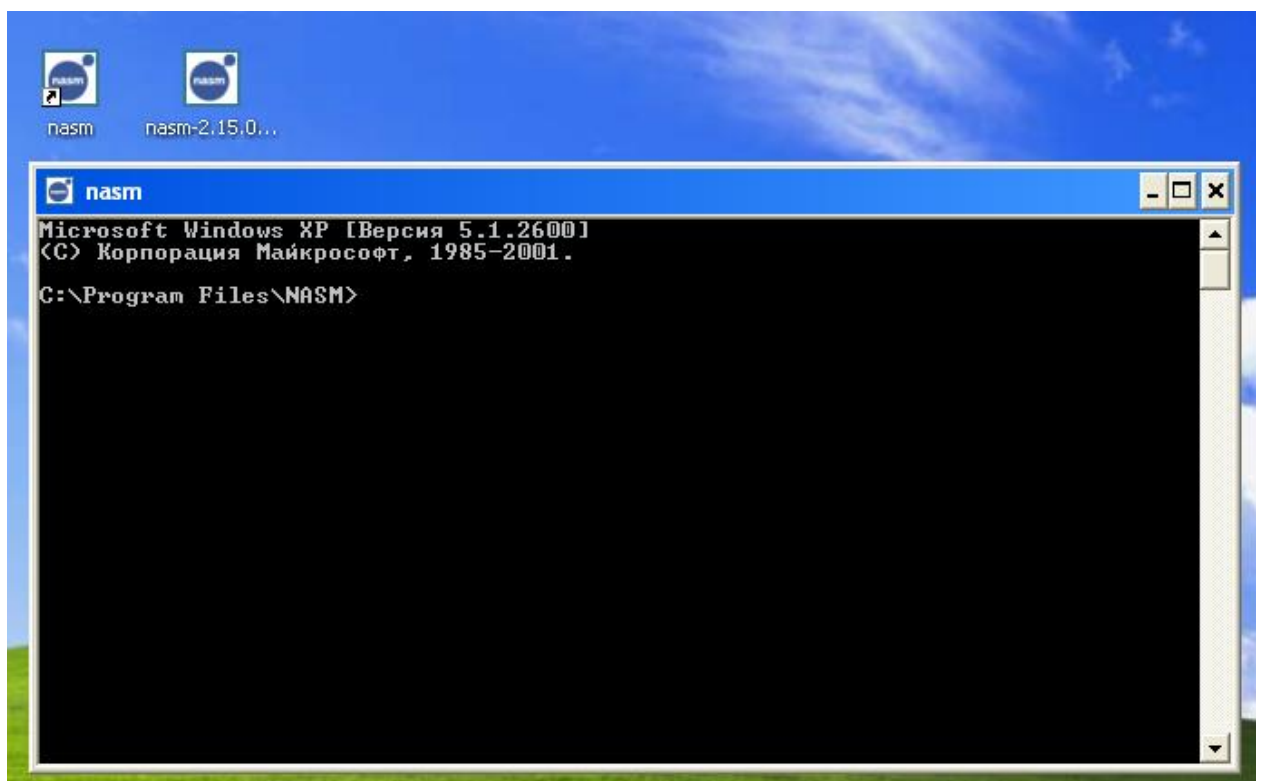
Установлена VirtualBox Guest Additions



Затем установлен двунаправленный режим функции Drag and Drop для переноса файлов в основную ОС из гостевой и обратно. Выполнена перезагрузка гостевой ОС.



В основной ОС была установлен NASM по [ссылке](#) для Windows 32 бит и перенесен на гостевую ОС. Произведена его установка.



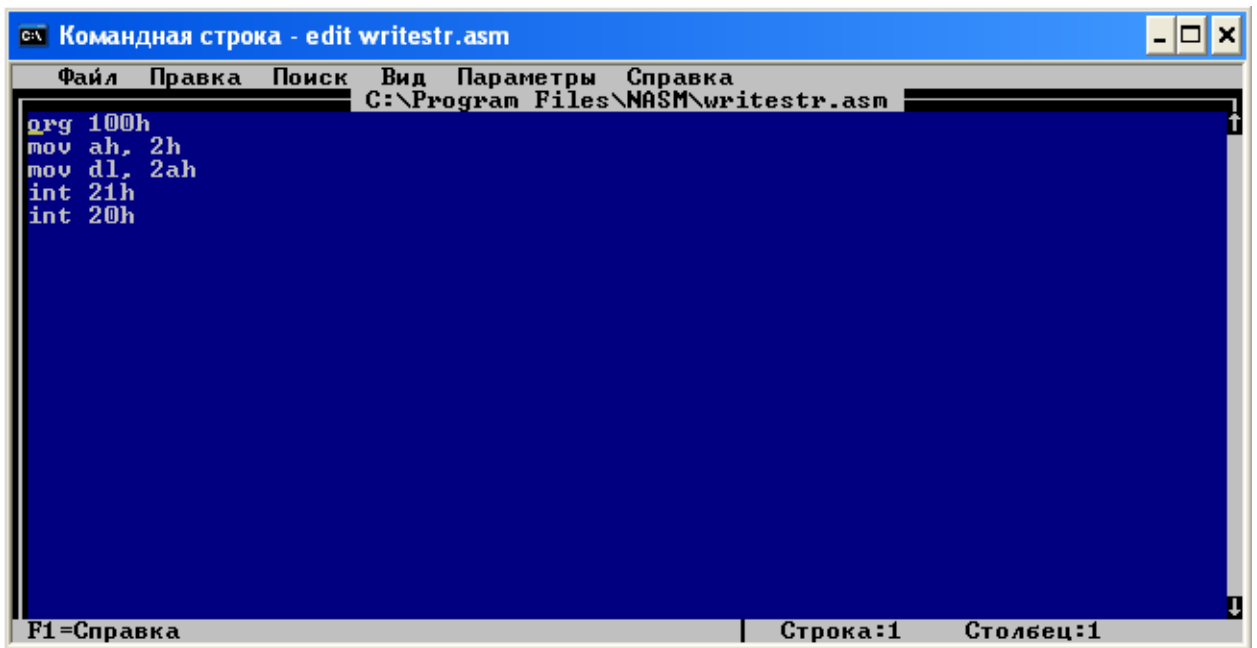
В файл writestr.asm с помощью редактора edit была записана исходная программа по выводу звездочки на экран. Затем проверено, что это ASCII-файл и проведено ассемблирование программы. И, наконец, осуществлен запуск программы.

```
C:\Program Files\NASM>edit writestr.asm
C:\PROGRA~1\NASM>type writestr.asm
org 100h
mov ah, 2h
mov dl, 2ah
int 21h
int 20h

C:\PROGRA~1\NASM>nasm writestr.asm -o writestr.com
nasm: error: unrecognised option '-o'

C:\PROGRA~1\NASM>nasm writestr.asm -o writestr.com

C:\PROGRA~1\NASM>writestr.com
*
```

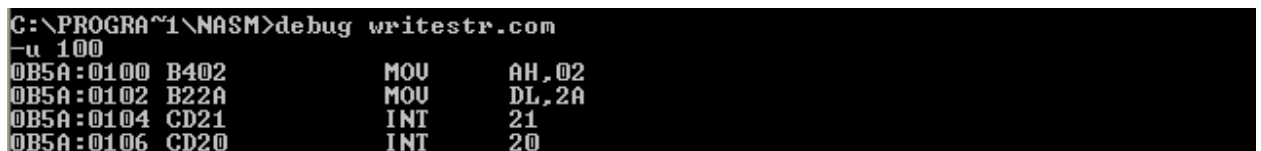


```
Командная строка - edit writestr.asm
Файл  Правка  Поиск  Вид  Параметры  Справка
C:\Program Files\NASM\writestr.asm

org 100h
mov ah, 2h
mov dl, 2ah
int 21h
int 20h

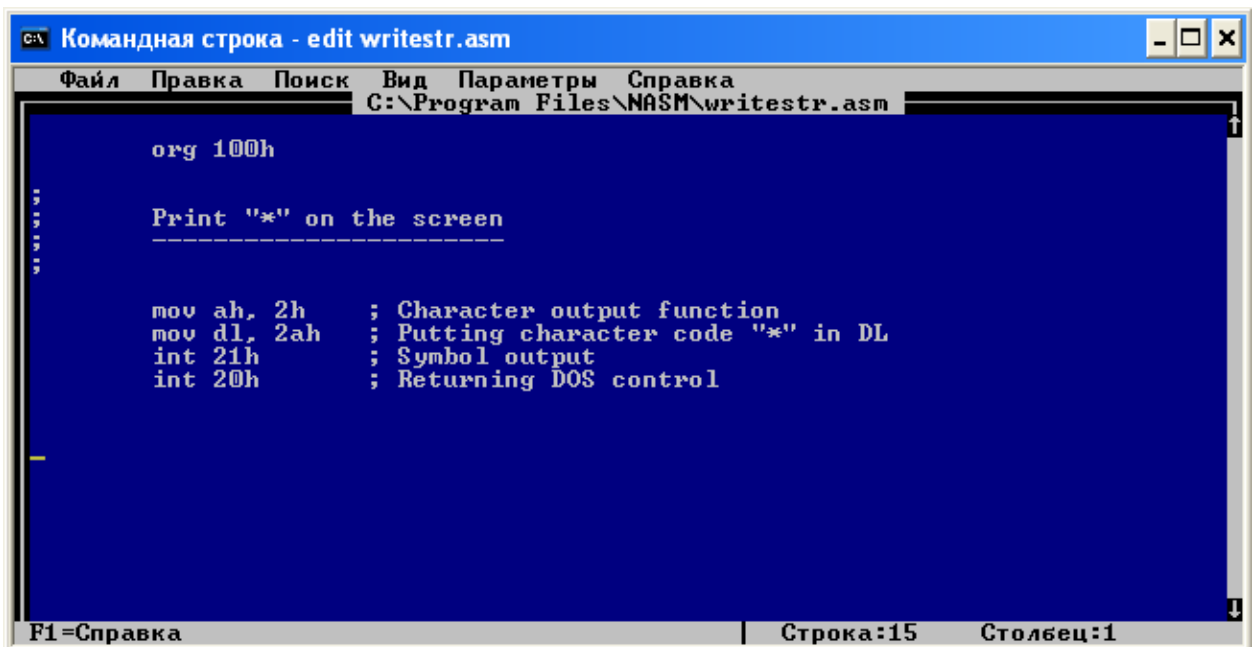
F1=Справка | Строка:1  Столбец:1
```

Также произведено разасемблирование файла в debug, чтобы увидеть ранее написанную программу



```
C:\PROGRA~1\NASM>debug writestr.com
-u 100
0B5A:0100 B402      MOV     AH,02
0B5A:0102 B22A      MOV     DL,2A
0B5A:0104 CD21      INT     21
0B5A:0106 CD20      INT     20
```

В программу были добавлены комментарии



```
Командная строка - edit writestr.asm
Файл  Правка  Поиск  Вид  Параметры  Справка
C:\Program Files\NASM\writestr.asm

    org 100h

    Print "*" on the screen
    -----

    mov ah, 2h      ; Character output function
    mov dl, 2ah     ; Putting character code "*" in DL
    int 21h         ; Symbol output
    int 20h         ; Returning DOS control

F1=Справка | Строка:15  Столбец:1
```

```

C:\PROGRA~1\NASM>type writestr.asm

    org 100h

;
;
; Print "*" on the screen
; -----

    mov ah, 2h    ; Character output function
    mov dl, 2ah   ; Putting character code "*" in DL
    int 21h       ; Symbol output
    int 20h       ; Returning DOS control

```

Также произведен ввод программы, которая была разработана ранее и осуществляет вывод на экран буквы от А до J

```

C:\PROGRA~1\NASM>edit printlet.asm
C:\PROGRA~1\NASM>nasm printlet.asm -o printlet.com
C:\PROGRA~1\NASM>printlet.com
ABCDEFGHIJ

```

```

Файл  Правка  Формат  Вид  Справка
org 100h
;
;      output 10 characters from A to J
; -----
; Calls: write_char
;
Start:
    mov dl,'A' ; In DL the code of the letter A
    mov cx,10  ; The character counter is -10
.Loop: call write_char ; Symbol output
    loop .Loop ; Move to next character
    int 20h    ; Return to DOS
;
;      Displaying a symbol on the screen and increasing the symbol code by 1
; -----
; Inputs: DL contains character code
; Outputs: DL contains the code for the new character
;
write_char:
    push ax
    mov ah,02 ; Character output function
    int 21h   ; Displaying a symbol on the screen
    inc dl    ; Next character code
    pop ax
    ret       ; Return from procedure

```

Просмотр разасемблированной программы в debug

```

C:\PROGRA~1\NASM>debug printlet.com
-u 100
0CD8:0100 B241      MOV     DL,41
0CD8:0102 B90A00    MOV     CX,000A
0CD8:0105 E80400    CALL    010C
0CD8:0108 E2FB      LOOP    0105
0CD8:010A CD20      INT     20
0CD8:010C 50        PUSH    AX
0CD8:010D B402      MOV     AH,02
0CD8:010F CD21      INT     21
0CD8:0111 FEC2      INC     DL
0CD8:0113 58        POP     AX
0CD8:0114 C3        RET

```

Лабораторная работа 8.

Вывод на экран двухзначного шестнадцатеричного числа с помощью программы на ассемблере.

Написанная и закомментированная программа:

```
org 100h

;

;   Тестирование процедуры Write_byte_hex
;   -----
;
; Вызовы: Write_byte_hex
;
Test_write_byte_hex:
    mov dl,3fh      ; Тестировать с 3Fh
    call Write_byte_hex ; Вывод числа
    int 20h         ; Возврат в DOS
;
;   Вывод двухзначного шестнадцатеричного числа
;   -----
;
; Входы: DL содержит выводимое число
; Вызовы: Write_digit_hex
;
Write_byte_hex:
;
; Вывод старшей цифры двухзначного шестнадцатеричного числа
;
    push cx
    push bx
    mov bl, dl ; Копирование значения из dl в bl
    mov cl, 04h ; Помещение 4 в cl для логического сдвига вправо на 4 бита
    shr bl, cl ; Логический сдвиг вправо для получения старшей цифры
    call Write_digit_hex ; Вывод старшей цифры
```

```

;
; Вывод младшей цифры двузначного шестнадцатеричного числа
;
    mov bl, dl ; Копирование значения из dl в bl
    and bl, 0fh ; Сравнение значения в bl с 0f для получения младшей цифры
    call Write_digit_hex ; Вывод младшей цифры
    pop bx
    pop cx
    ret
;
; Вывод шестнадцатеричной цифры
; -----
;
; Входы: BL содержит шестнадцатеричную цифру
; Вызовы: Write_char
;
Write_digit_hex:
    push dx
    mov dl,bl ; Копирование значения из bl в dl
    cmp dl,09h ; Различная обработка цифр от 0 до 9 и от A до F
    ja .ifg ; Если цифра больше 9, то совершить переход
    add dl,30h ; Прибавить 30 к цифре от 0 до 9
    jmp .pass ; Пропуск следующей строки
.ifg: add dl,37h ; Прибавить 37 к цифре от A до F
.pass: call Write_char ; Вызов функции по выводу символа
    pop dx
    ret
;
; Вывод символа на экран
; -----
;
; Входы: DL содержит код символа, выводимый на экран

```

;

Write_char:

```
    push ax
    mov ah, 02h ; Функция вывода символа
    int 21h ; Вывод символа на экран
    pop ax
    ret
```

```
    org 100h
;
;   Тестирование процедуры write_byte_hex
;   -----
;
;   Вызовы: write_byte_hex
;
Test_write_byte_hex:
    mov dl, 3fh ; Тестировать с 3Fh
    call write_byte_hex ; Вывод числа
    int 20h ; Возврат в DOS
;
;   Вывод двузначного шестнадцатеричного числа
;   -----
;
;   Входы: DL содержит выводимое число
;   Вызовы: write_digit_hex
;   bl
write_byte_hex:
;
;   Вывод старшей цифры двузначного шестнадцатеричного числа
;
    push cx
    push bx
    mov bl, dl ; копирование значения из dl в bl
    mov cl, 04h ; Помещение 4 в cl для логического сдвига вправо на 4 бита
    shr bl, cl ; логический сдвиг вправо для получения старшей цифры
    call write_digit_hex ; Вывод старшей цифры
;
;   Вывод младшей цифры двузначного шестнадцатеричного числа
;
    mov bl, dl ; копирование значения из dl в bl
    and bl, 0fh ; Сравнение значения в bl с 0f для получения младшей цифры
    call write_digit_hex ; Вывод младшей цифры
    pop bx
    pop cx
    ret
```

```

;
;      Вывод шестнадцатеричной цифры
;      -----
;
; Входы: BL содержит шестнадцатеричную цифру
; Вызовы: write_char
;
write_digit_hex:
    push dx
    mov dl,bl          ; Копирование значения из bl в dl
    cmp dl,09h         ; Различная обработка цифр от 0 до 9 и от A до F
    ja .ifg           ; Если цифра больше 9, то совершить переход
    add dl,30h         ; Прибавить 30 к цифре от 0 до 9
    jmp .pass         ; Пропуск следующей строки
.ifg: add dl,37h         ; Прибавить 37 к цифре от A до F
.pass: call write_char ; Вызов функции по выводу символа
    pop dx
    ret
;
;      Вывод символа на экран
;      -----
;
; Входы: DL содержит код символа, выводимый на экран
;
write_char:
    push ax
    mov ah, 02h ; Функция вывода символа
    int 21h     ; Вывод символа на экран
    pop ax
    ret

```

Получение файла и проверка работы программы

```

C:\Program Files\NASM>nasm Video_io.asm -o Video_io.com
C:\Program Files\NASM>Video_io.com
3F

```

Тестирование программы

```

C:\PROGRA~1\NASM>debug Video_io.com
-u 100
0B5D:0100 B23F      MOV     DL,3F
0B5D:0102 E80200    CALL    0107
0B5D:0105 CD20      INT     20
0B5D:0107 51        PUSH    CX
0B5D:0108 53        PUSH    BX
0B5D:0109 88D3      MOV     BL,DL
0B5D:010B B104      MOV     CL,04
0B5D:010D D2EB      SHR     BL,CL
0B5D:010F E80B00    CALL    011D
0B5D:0112 88D3      MOV     BL,DL
0B5D:0114 80E30F    AND     BL,0F
0B5D:0117 E80300    CALL    011D
0B5D:011A 5B        POP     BX
0B5D:011B 59        POP     CX
0B5D:011C C3        RET
0B5D:011D 52        PUSH    DX
0B5D:011E 88DA      MOV     DL,BL

```



```
-g
3F
Программа завершилась нормально
-e 101
0B5D:0101  3F.00
-g
00
Программа завершилась нормально
-e 101
0B5D:0101  00.09
-g
09
Программа завершилась нормально
-e 101
0B5D:0101  09.0A
-g
0A
Программа завершилась нормально
-e 101
0B5D:0101  0A.FF
-g
FF
Программа завершилась нормально
```